HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



Đề tài: Hệ thống IoT giám sát môi trường và điều khiển thiết bị

BÁO CÁO BÀI TẬP LỚN

Giảng viên	**:**	**TS. Nguyễn Quốc Uy**
Môn học	**:**	**IoT và ứng dụng**
Nhóm lớp	**:**	**10**
Họ và tên	**:**	**Hoàng Quốc Anh**
Mã sinh viên	**:**	**B23DCCN022**

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1 GIỚI THIỆU 3
1.1 Bài toán và mục tiêu 3
1.2 Phạm vi thực hiện 3
1.3 Phương pháp và bố cục 3
CHƯƠNG 2 PHÂN TÍCH 4
2.1 Tác nhân và chức năng 4
2.2 Dữ liệu và quy tắc nghiệp vụ 5
2.3 Yêu cầu phi chức năng 5
CHƯƠNG 3 THIẾT KẾ 6
3.1 Kiến trúc và môi trường triển khai 6
3.2 Phần cứng và firmware 7
3.3 Cơ sở dữ liệu 8
3.4 Đăng nhập 9
3.5 Dashboard và dữ liệu realtime 10
3.6 Điều khiển LED có xác nhận 12
3.7 Dữ liệu cảm biến 14
3.8 Lịch sử điều khiển 15
3.9 Hồ sơ người dùng 17
3.10 Tài liệu API và vận hành 18
CHƯƠNG 4 KẾT LUẬN 19
4.1 Kết quả đạt được 19
4.2 Hạn chế 19
4.3 Hướng phát triển 19
TÀI LIỆU THAM KHẢO 20
PHỤ LỤC TỪ ĐIỂN DỮ LIỆU 21

CHƯƠNG 1 GIỚI THIỆU

1.1 Bài toán và mục tiêu

Đề tài xây dựng hệ thống Web theo dõi nhiệt độ, độ ẩm, ánh sáng và điều khiển hai LED từ ESP32. Số đo được lưu vào MySQL, hiển thị realtime và có thể tra cứu. Mỗi lệnh điều khiển được ghi người thực hiện, theo dõi xác nhận và thông báo khi không có phản hồi.

Bảng 1.1 Mục tiêu đề tài

Mục tiêu	Kết quả cần đạt
Thu thập	Đọc DHT11 và LDR; truyền số đo qua MQTT.
Giám sát	Hiển thị số đo cuối, một biểu đồ ba đường và trạng thái phần cứng.
Điều khiển	ON/OFF hai LED; chỉ cập nhật trạng thái theo xác nhận hợp lệ.
Tra cứu	Tìm kiếm, lọc, sắp xếp và phân trang cảm biến/lịch sử.
Người dùng	Đăng nhập bằng JWT; xem/sửa hồ sơ và ghi username cho từng lệnh.
Triển khai	Khởi động dịch vụ bằng Docker; cung cấp Swagger/OpenAPI.

1.2 Phạm vi thực hiện

Phần cứng gồm ESP32, DHT11, LDR và hai LED trong mạng Wi-Fi nội bộ. Ứng dụng có năm màn hình: Đăng nhập, Dashboard, Dữ liệu cảm biến, Lịch sử điều khiển và Hồ sơ. Ba tài khoản user123, user124, user125 có cùng nhóm chức năng; không có đăng ký hoặc phân quyền nhiều vai trò.

Lux được ước lượng từ ADC, chưa hiệu chuẩn. LED là thiết bị chấp hành của mô hình; phản hồi MQTT xác nhận mức GPIO theo firmware. Hệ thống chưa điều khiển thiết bị điện lưới.

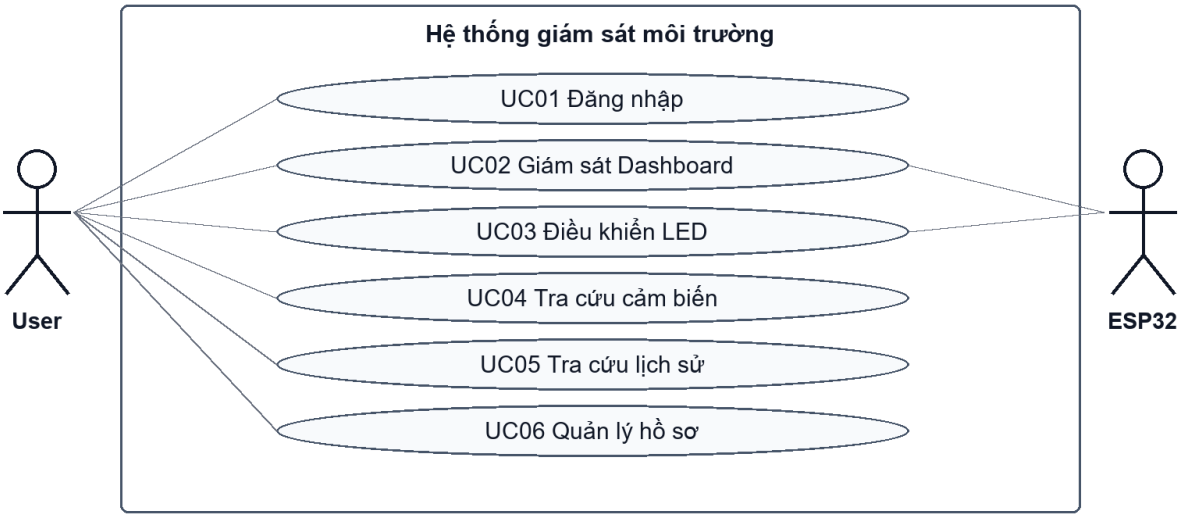
1.3 Phương pháp và bố cục

Quá trình thực hiện gồm phân tích từ TH1, thiết kế dữ liệu/giao diện, xây dựng ứng dụng, tích hợp ESP32 và kiểm tra xuyên suốt MQTT–Backend–MySQL–WebSocket–Web. Báo cáo gồm bốn chương: Giới thiệu, Phân tích, Thiết kế và Kết luận. Kết quả kiểm thử sử dụng lượt chốt ngày 27/10/2026 [1], [2].

CHƯƠNG 2 PHÂN TÍCH

2.1 Tác nhân và chức năng

User sử dụng Web; ESP32 gửi số đo và phản hồi trạng thái. Các chức năng sau đăng nhập dùng chung giữa ba tài khoản, riêng hồ sơ được truy xuất theo chủ thể JWT.



Hình 2.1 Các use case và tác nhân của hệ thống

Bảng 2.1 Yêu cầu chức năng

Mã	Chức năng	Kết quả
UC01	Đăng nhập	Xác thực tài khoản và cấp JWT.
UC02	Giám sát Dashboard	Số đo cuối, biểu đồ và tình trạng phần cứng.
UC03	Điều khiển LED	Gửi ON/OFF; chờ xác nhận hoặc timeout.
UC04	Tra cứu cảm biến	Tìm theo đại lượng/giá trị/thời gian, sort/page.
UC05	Tra cứu điều khiển	Lọc lệnh và xem đúng username người gửi.
UC06	Quản lý hồ sơ	Xem, sửa, lưu hoặc hủy thay đổi.

2.2 Dữ liệu và quy tắc nghiệp vụ

Bảng 2.2 Danh mục cảm biến

Mã nguồn đo	Đại lượng	Đơn vị
DHT11_TEMP	Nhiệt độ từ DHT11	°C
DHT11_HUM	Độ ẩm từ DHT11	%RH
LDR_LIGHT	Ánh sáng quy đổi từ ADC	lux ước lượng

Bảng 2.3 Quy tắc chính

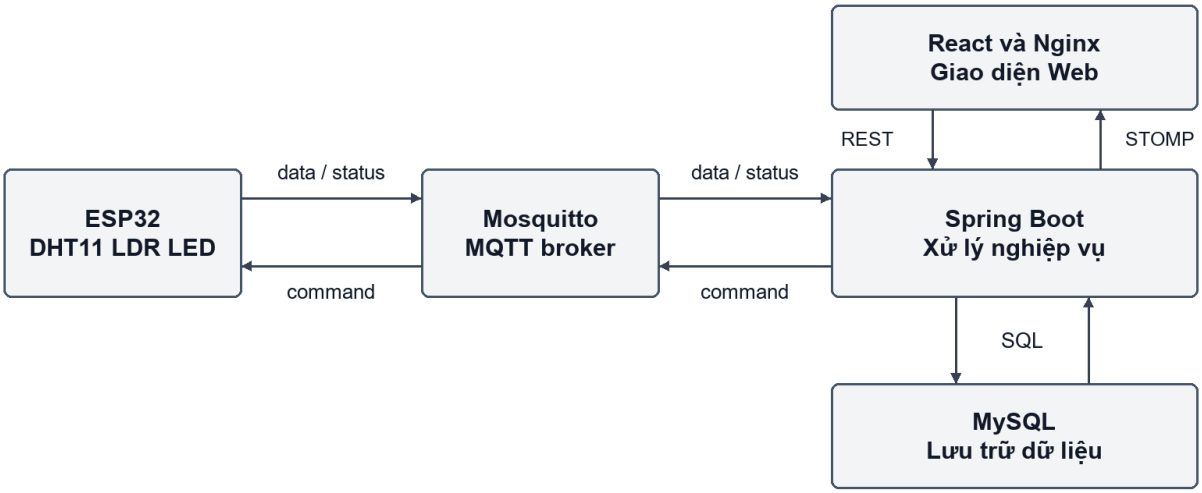
Quy tắc	Nội dung
Bản ghi	Một reading hợp lệ tạo một SensorData; không có sample gộp ba cảm biến.
Biểu đồ	Tối đa 15 reading mới nhất mỗi Sensor. X thể hiện thứ tự, nhãn giờ theo nhiệt độ; tooltip giữ giờ riêng từng Sensor.
Kết nối	Không có telemetry hợp lệ trong 30 giây thì Offline; giữ số đo cuối.
Lệnh LED	Mỗi lần bấm tạo ActionHistory, kể cả lệnh trùng trạng thái. requestId = ActionHistory.id.
Xác nhận	202 chỉ là tiếp nhận. Chỉ status MQTT hợp lệ mới xác nhận lịch sử và cập nhật Device.
Timeout	Sau 10 giây chưa có xác nhận: TIMEOUT, Device giữ trạng thái đã xác nhận gần nhất.
Processing	Hiển thị khi deliveryState=PENDING; cột action/status trong SQL vẫn chỉ ON/OFF.
Người gửi	Lấy User.id từ JWT; hiển thị User.username qua quan hệ lịch sử, không nhận actor từ body.
Truy vấn	UI mặc định 5 dòng; cho chọn 5/10/15/20. Filter trước sort và page.
Thời gian	Lưu và truyền JSON theo UTC; giao diện hiển thị giờ Việt Nam.

2.3 Yêu cầu phi chức năng

API nghiệp vụ cần JWT; mật khẩu lưu BCrypt. Sự kiện realtime được phát sau khi dữ liệu commit. Giao diện thể hiện trạng thái tải/rỗng/lỗi/chờ, không tự tạo số đo hoặc giả lập ACK. Sau kết nối lại, Frontend tải snapshot REST để phục hồi trạng thái; cấu hình và secrets được tách khỏi source [2].

CHƯƠNG 3 THIẾT KẾ

3.1 Kiến trúc và môi trường triển khai



Hình 3.1 Kiến trúc và chiều trao đổi dữ liệu

MQTT kết nối phần cứng với Backend theo mô hình publish/subscribe [3]. REST tải snapshot, tra cứu và tiếp nhận thao tác. STOMP trên WebSocket đẩy thay đổi tới React [4]. Frontend không truy cập trực tiếp MySQL hoặc broker.

Bảng 3.1 Môi trường hiện tại

Thành phần	Triển khai và kết nối
React + Nginx + Spring Boot	Container app; Web tại localhost:8088. Nginx proxy /api và /ws tới Backend:8080.
Mosquitto	Service trong Compose. Backend: mosquitto:1883; ESP32: IP LAN laptop:1325.
MySQL	Container sẵn có, volume được giữ. Backend: host.docker.internal:3307; MySQL nội bộ:3306.
ESP32	Kết nối Wi-Fi và MQTT thật; không dùng localhost hoặc tên service Docker.

Compose quản lý app và broker; MySQL hiện là container riêng. start.ps1 khởi động MySQL hiện có trước Compose. Chế độ BTH4 tắt IOT_LED_DEMO_ENABLED; không có responder mô phỏng trạng thái LED.

3.2 Phần cứng và firmware



Hình 3.2 Ánh xạ GPIO theo sketch của dự án

Bảng 3.2 Kết nối thiết kế

Tín hiệu	GPIO	Chức năng
DHT11 DATA	4	Đọc nhiệt độ và độ ẩm.
LDR AO	34	Đọc ADC analog.
LED1	18	Điều khiển LED 1 qua điện trở.
LED2	19	Điều khiển LED 2 qua điện trở.

Firmware khởi tạo GPIO và cảm biến, kết nối Wi-Fi/MQTT rồi lặp xử lý lệnh và đọc cảm biến theo cấu hình khoảng 2 giây. LDR lấy trung bình 10 mẫu ADC; hàm adcToLux quy đổi xấp xỉ 0–200 lux. analogRead cung cấp giá trị ADC, không tự tạo phép đo lux đã hiệu chuẩn [7].

Khi nhận command, ESP32 kiểm tra requestId/deviceCode/action, đổi GPIO và publish status cùng requestId. Các gói của ba đại lượng được gửi riêng; thời gian Backend nhận có thể khác nhau.

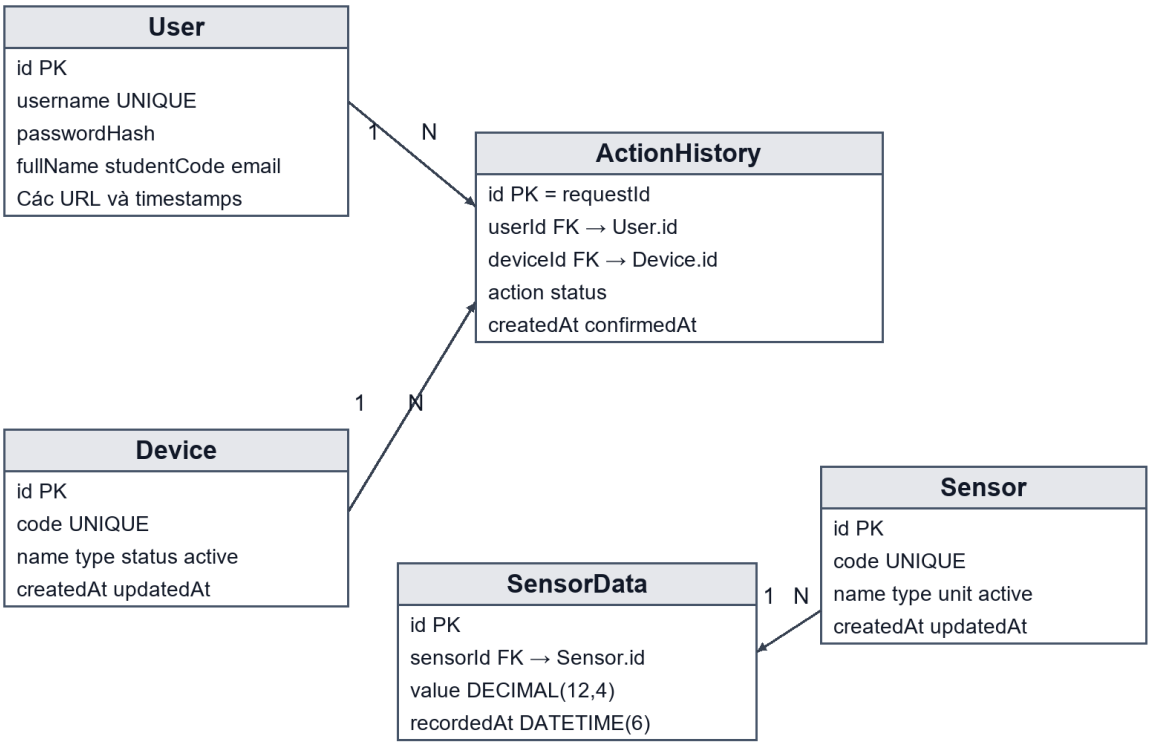
Chèn ảnh mạch toàn cảnh và cận chân nối tại đây.

Hình 3.3 Mạch thực tế của hệ thống

Chèn ảnh Serial Monitor thể hiện số đo và phản hồi LED tại đây.

Hình 3.4 Số đo và phản hồi trên Serial Monitor

3.3 Cơ sở dữ liệu



Hình 3.5 ERD rút gọn gồm năm bảng

Bảng 3.3 Quan hệ dữ liệu

Quan hệ	Ý nghĩa
Sensor 1–N SensorData	Một Sensor có nhiều reading.
User 1–N ActionHistory	Một User gửi nhiều lệnh.
Device 1–N ActionHistory	Một LED có nhiều yêu cầu điều khiển.

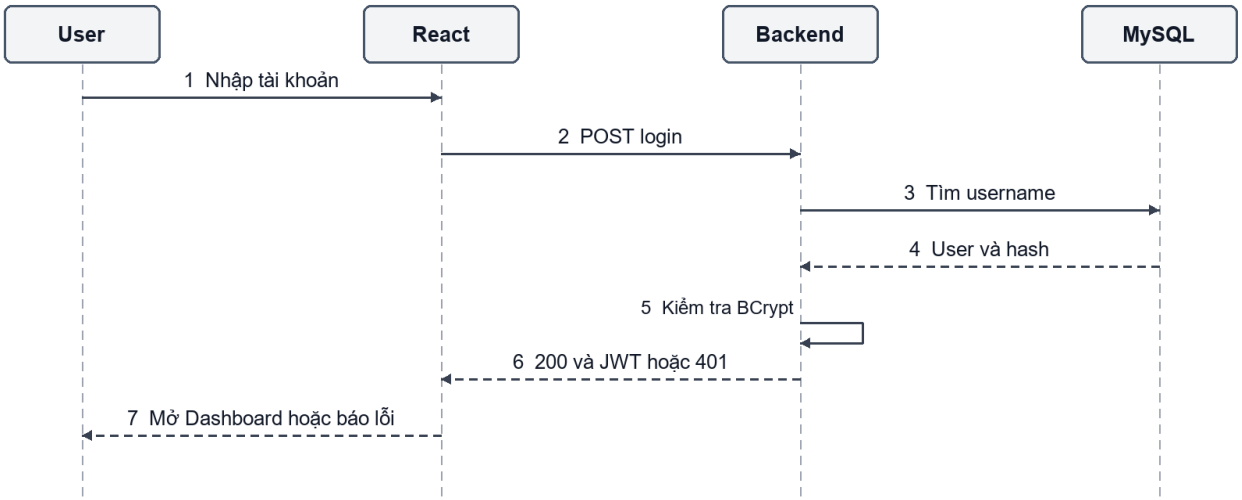
Schema gồm năm bảng, 40 cột. SensorData.value dùng DECIMAL(12,4), thời gian dùng DATETIME(6) theo UTC [6]. Chỉ mục ghép (sensorId, recordedAt) và (deviceId, createdAt) hỗ trợ tra cứu. Từ điển đầy đủ đặt tại Phụ lục.

ActionHistory lưu userId/deviceId/action/status/createdAt/confirmedAt. confirmedAt=NULL khi chưa xác nhận; deliveryState được suy ra theo thời gian. Username trả qua JOIN User, không thêm cột username vào ActionHistory. Seed chỉ thêm tài khoản thiếu, không ghi đè hồ sơ hoặc mật khẩu đã có.

3.4 Đăng nhập



Hình 3.6 Giao diện đăng nhập



Hình 3.7 Sequence đăng nhập

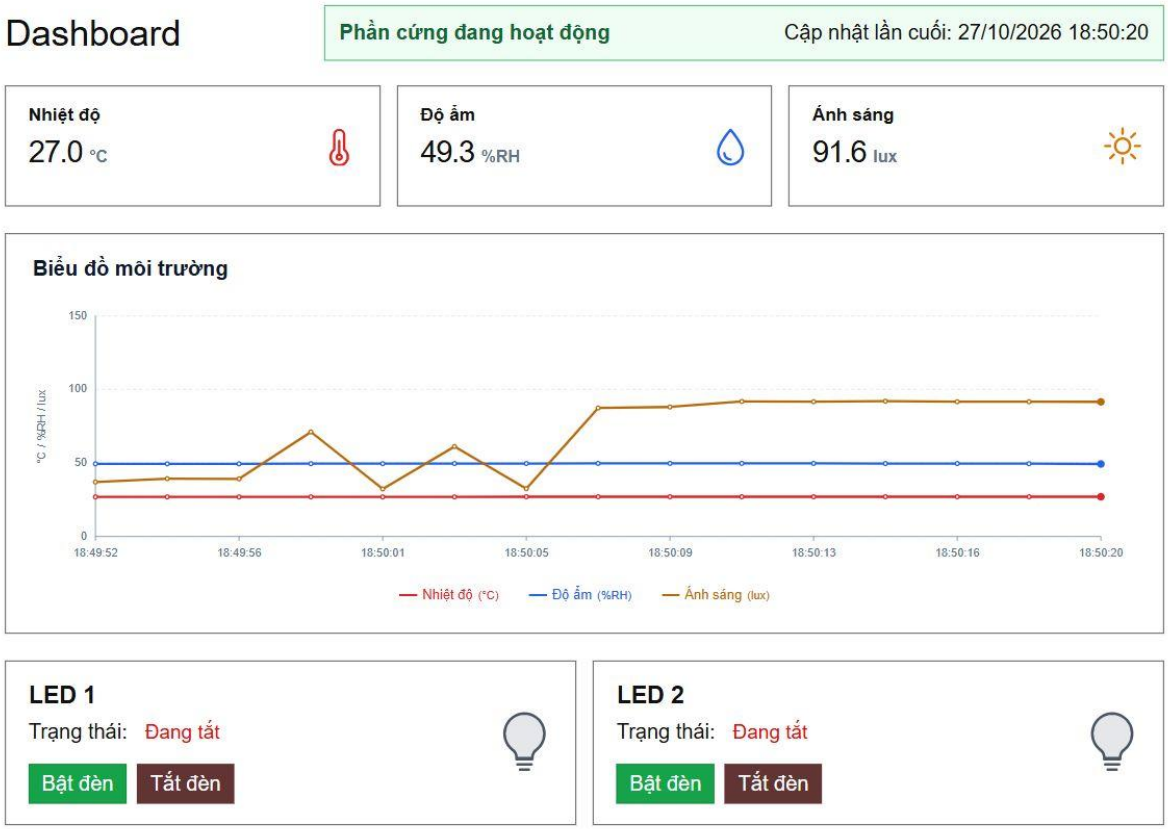
Bảng 3.4 API đăng nhập

API	Dữ liệu và phản hồi
POST /api/auth/login	Body: username, password. Hợp lệ: 200 {token}; sai tài khoản/mật khẩu:401; thiếu trường:400.

JWT dùng HS256, issuer iot-backend, subject User.id; REST gửi Authorization: Bearer. Token được kiểm tra chữ ký và hạn dùng; Frontend lưu phiên trong localStorage và chuyển Login khi hết hạn/401. Thời hạn mặc định 3.600 giây, có thể cấu hình [2].

Kiểm tra 27/10: cả ba tài khoản đăng nhập được; mật khẩu sai, token thiếu/sai/hết hạn bị từ chối. UI đăng nhập lại thành công sau khi phiên cũ hết hạn.

3.5 Dashboard và dữ liệu realtime



Hình 3.8 Dashboard Online sau kiểm tra với ESP32

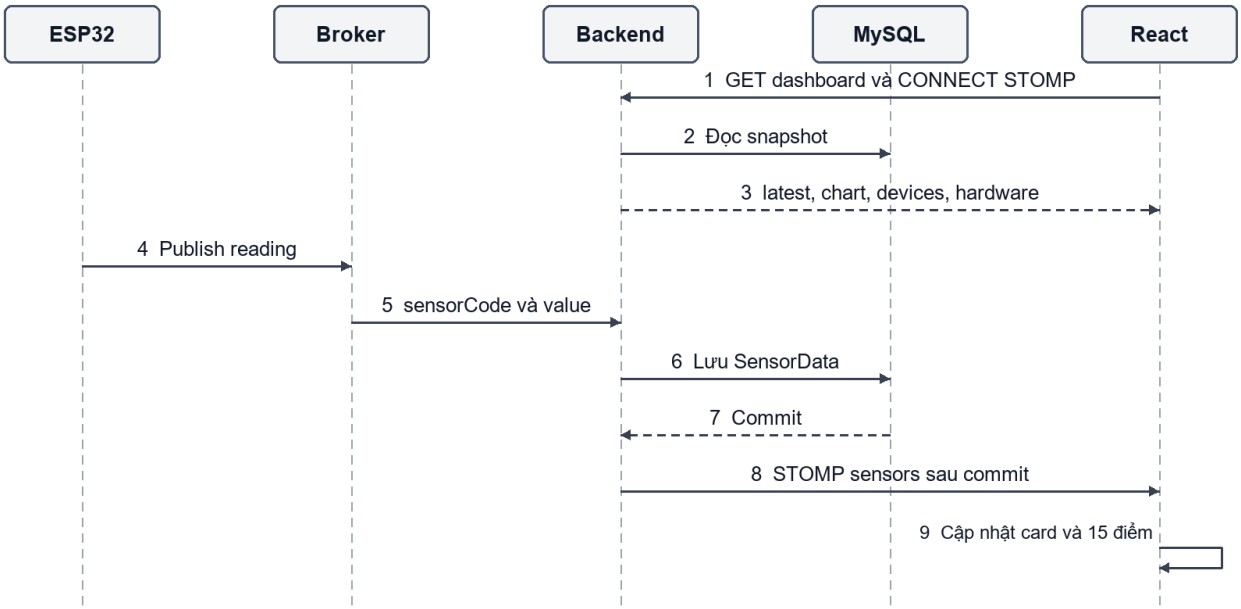
Dashboard tải GET /api/dashboard rồi đăng ký realtime bằng useEffect; cleanup hủy listener khi rời trang [5]. Khi STOMP connected/reconnected, tải lại snapshot. Event sensors cập nhật reading/card; devices cập nhật LED; hardware đổi banner; notifications xử lý timeout.

Bảng 3.5 Ánh xạ ba đường biểu đồ

Đường	Mã dữ liệu	Đơn vị
Đỏ	DHT11_TEMP	°C
Xanh	DHT11_HUM	%RH
Vàng	LDR_LIGHT	lux

Mỗi đường giữ 15 reading mới nhất, X cách đều từ vị trí 1 đến 15. Nhãn giờ lấy từ nhiệt độ; tooltip giữ thời gian thật từng Sensor. Khi đủ dữ liệu, ba đường thẳng hàng hai đầu; không nội suy giá trị hay ghép thành chu kỳ đo chung.

Luồng dữ liệu và hợp đồng giao tiếp



Hình 3.9 Sequence từ ESP32 tới Dashboard

Bảng 3.6 Ba topic MQTT của dự án

Topic MQTT	JSON	Chiều
iot/sensor/data	sensorCode, value	ESP32 → Backend
iot/device/command	requestId, deviceCode, action	Backend → ESP32
iot/device/status	requestId, deviceCode, status	ESP32 → Backend

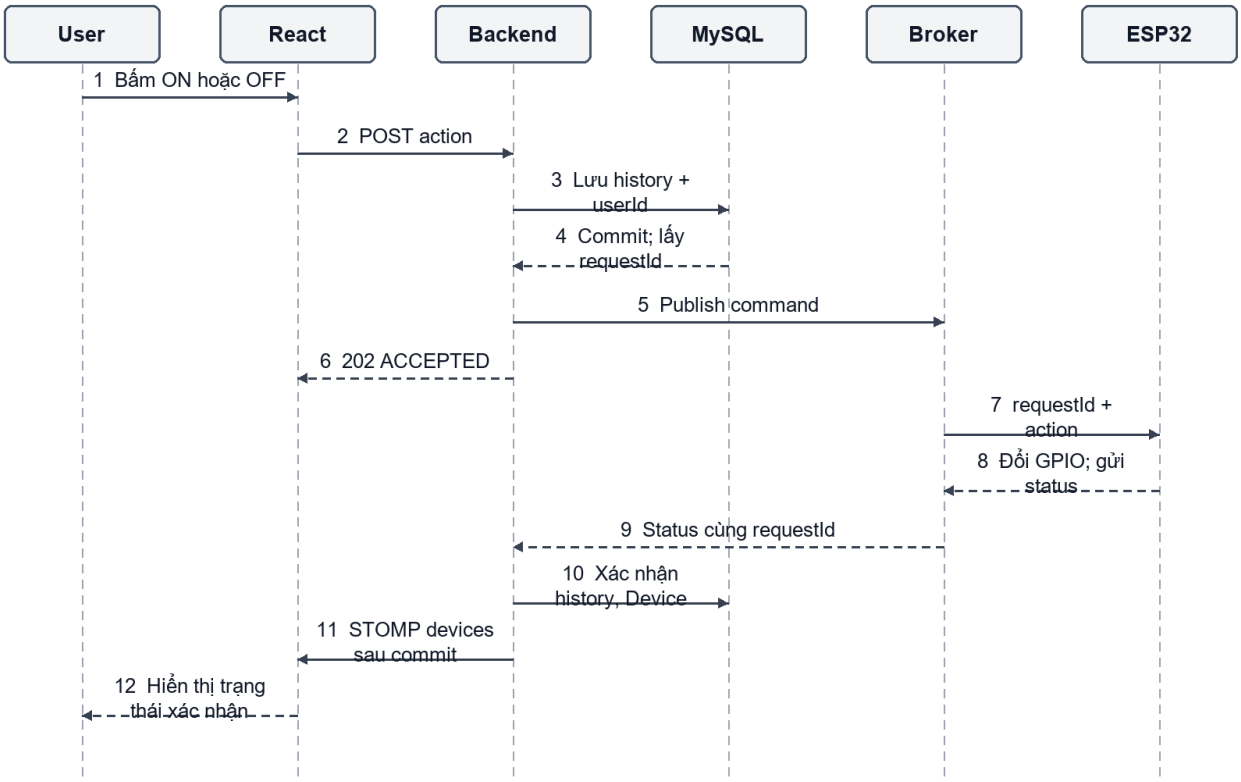
MQTT dùng QoS 0, không retain; Backend bỏ gói telemetry/status retained và payload không hợp lệ. Telemetry chỉ nhận Sensor đã đăng ký, active, giá trị hữu hạn và đúng miền. recordedAt do Backend gán, không phải thời điểm lấy mẫu trên ESP32.

Bảng 3.7 Các kênh realtime

STOMP destination	Dữ liệu gửi tới Frontend
/topic/sensors	Một SensorData đã commit, gồm id/mã/giá trị/đơn vị/giờ.
/topic/hardware	ONLINE/OFFLINE và lastSeenAt.
/topic/devices	Trạng thái Device hiện tại và request đã xác nhận.
/topic/notifications	DEVICE_TIMEOUT và requestId.

GET /api/dashboard trả latest, chart, devices và hardware. JWT được gửi ở STOMP CONNECT tại /ws; chỉ cho subscribe bốn topic trên, không nhận client SEND. Lượt chốt: 522 MQTT reading = 522 STOMP = 522 SQL khớp, 174 mỗi Sensor; Dashboard tự Offline/Online khi rút/cắm [2].

3.6 Điều khiển LED có xác nhận



Hình 3.10 Sequence điều khiển LED có requestId

POST /api/devices/{deviceId}/actions; body chỉ có action=ON hoặc OFF. Backend lấy User từ JWT, lưu history rồi publish command. HTTP 202 trả requestId và status=ACCEPTED; publish thất bại trả 503 kèm requestId, history vẫn được giữ.

```
{"requestId":617,"deviceCode":"LED1","action":"ON"}
{"requestId":617,"deviceCode":"LED1","status":"ON"}
```

Bảng 3.8 Vòng đời yêu cầu điều khiển

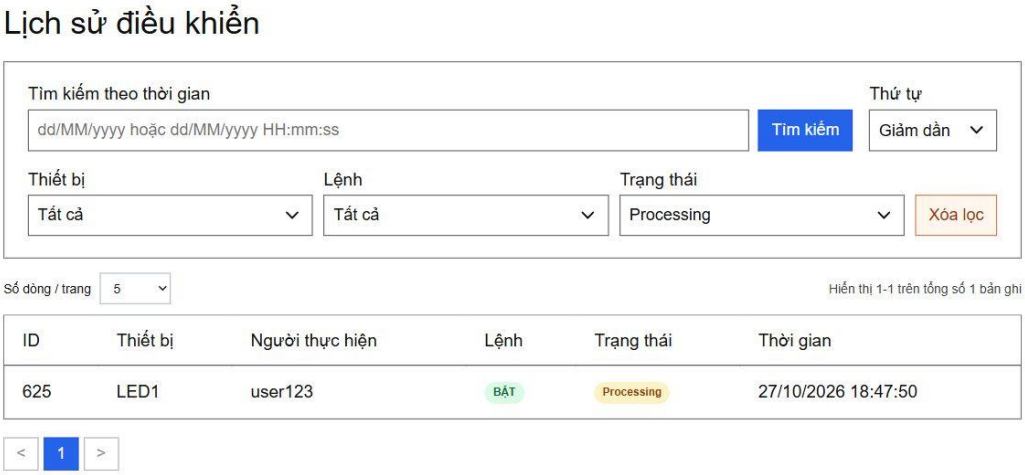
deliveryState	Điều kiện	Tác động
PENDING	Chưa xác nhận, dưới 10 giây.	Hiện Processing; giữ Device.status.
CONFIRMED	Status hợp lệ đúng requestId/deviceCode.	Ghi confirmedAt, cập nhật theo ACK.
TIMEOUT	Chưa ACK sau 10 giây.	Báo lỗi; không tự đổi đèn.

Status lặp không đổi xác nhận đầu; ACK cũ không ghi đè Device nếu request mới hơn đã xác nhận. ACK đến muộn có thể hoàn tất history. Đây là quy tắc xử lý trong code; lượt phần cứng cuối chưa thử ACK muộn/lặp/đảo thứ tự.

Giao diện và kết quả điều khiển



Hình 3.11 Chờ bật màu xanh và chờ tắt màu đỏ



Hình 3.12 Request 625 đang Processing khi chưa có xác nhận



Hình 3.13 Timeout giữ nguyên trạng thái đã xác nhận

Bảng 3.9 Các yêu cầu chủ động ngày 27/10

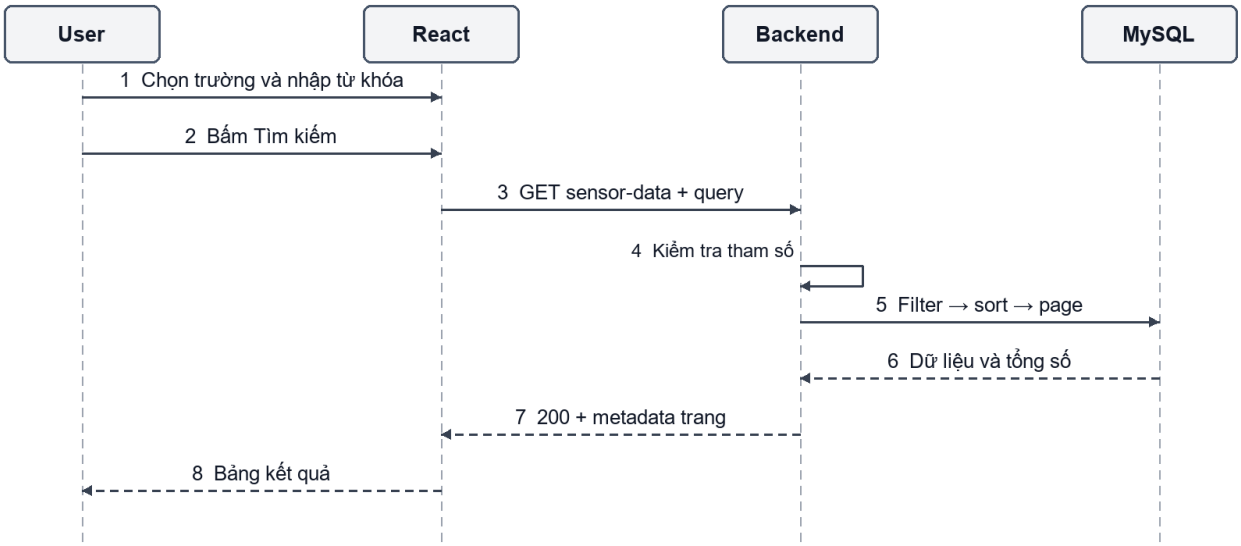
Request	Người gửi và thao tác	Kết quả
617–618	user123; LED1 ON/OFF qua REST.	CONFIRMED
619–620	user124; LED2 ON/OFF qua REST.	CONFIRMED
621–622	user125; LED1 ON/OFF qua REST.	CONFIRMED
623–626	user123; Web khi đã rút ESP32.	TIMEOUT
627–628	user123; Web LED2 ON/OFF sau cắm lại.	CONFIRMED

Tám xác nhận được đối chiếu MQTT command/status, STOMP, SQL và REST; bốn timeout không có status, confirmedAt=NULL. Thông báo pending không có nền; nút thao tác tiếp theo sáng hơn. Kết thúc hai Device OFF [2].

3.7 Dữ liệu cảm biến



Hình 3.14 Tìm đúng một giây trên dữ liệu thật



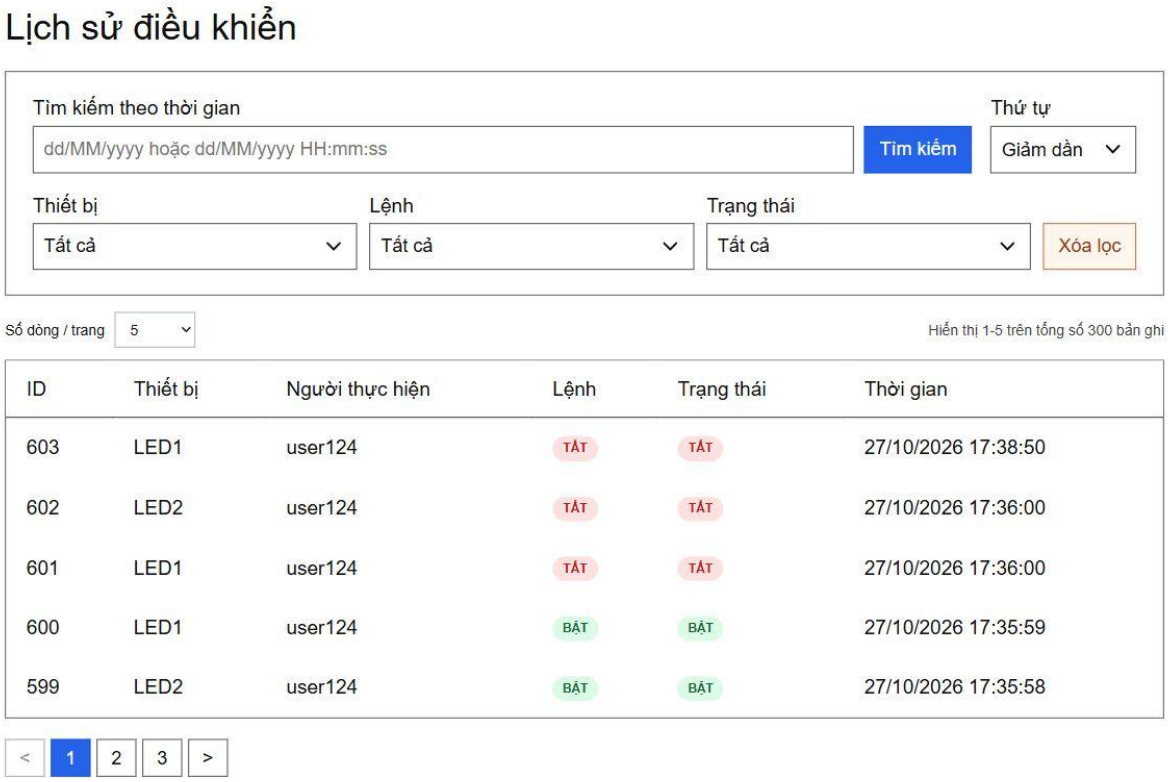
Hình 3.15 Sequence tra cứu dữ liệu cảm biến

GET /api/sensor-data nhận page, size, sortBy, order, search, sensorType, from/to. Chọn dropdown Tìm theo chỉ chuẩn bị trường tìm kiếm; bấm Tìm kiếm mới áp dụng. Chọn Thời gian sẽ đổi ngày/giây theo giờ Việt Nam thành khoảng UTC.

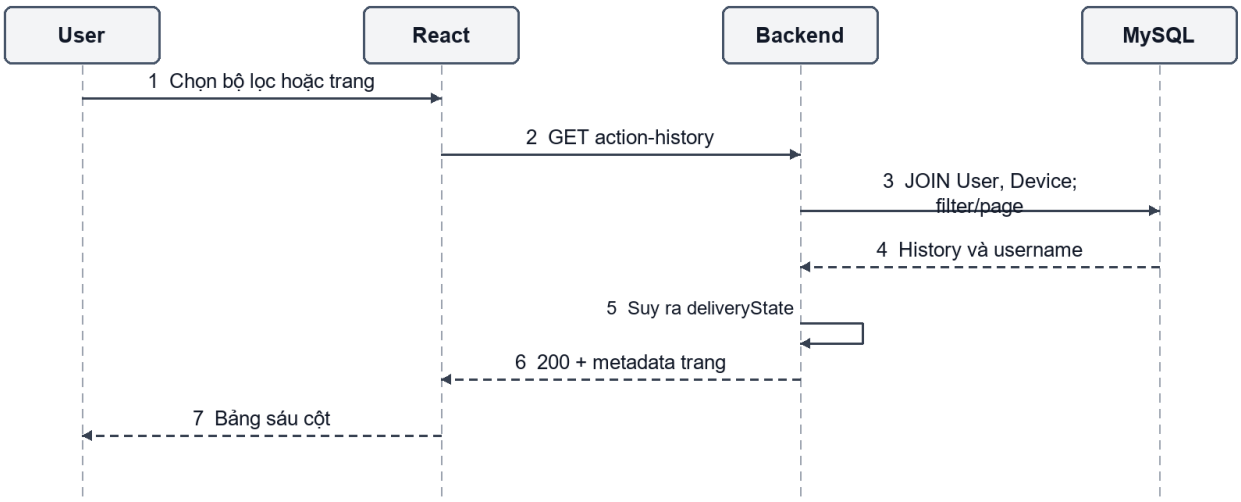
Response: {content, page, size, totalElements, totalPages}; mỗi hàng gồm id, sensorCode, sensorType, value, unit, recordedAt. Filter trước sort/page. UI cho 5/10/15/20 dòng, số dòng ở trái và tổng số ở phải; đổi thứ tự hoặc điều kiện đưa về trang đầu.

Kiểm tra: đổi dropdown chưa lọc ngay; tìm 27.5 chỉ trả nhiệt độ phù hợp; tìm 27/10/2026 18:44:00 trả ba reading đúng giây. Realtime gom tín hiệu tải lại trong 300 ms, giữ bảng trong lúc chờ [2].

3.8 Lịch sử điều khiển



Hình 3.16 Lịch sử có Người thực hiện và đầy đủ ngày giờ



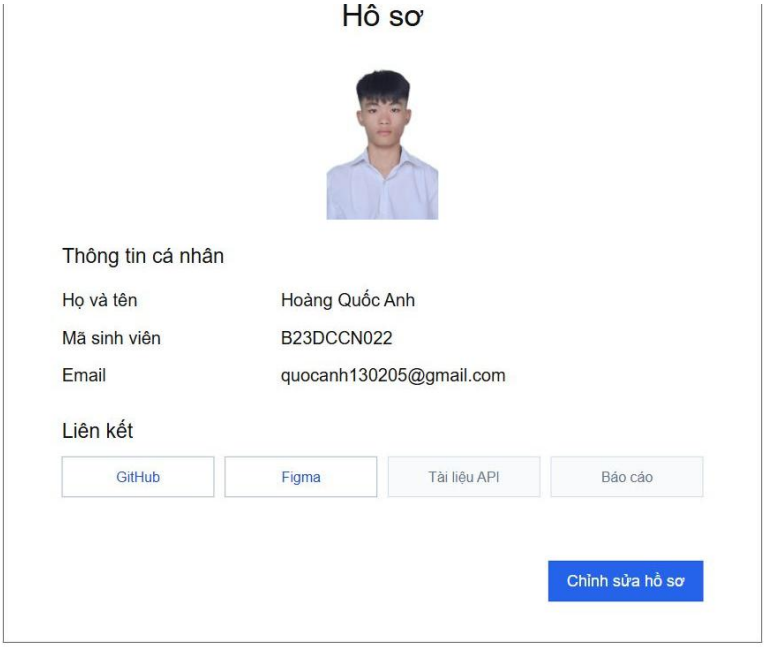
Hình 3.17 Sequence truy vấn lịch sử và username

GET /api/action-history nhận page/size/sortBy/order, search, device, action, status, from/to. Lọc Processing gửi status=PROCESSING, tương ứng deliveryState=PENDING. Xóa lọc đưa các điều kiện về mặc định.

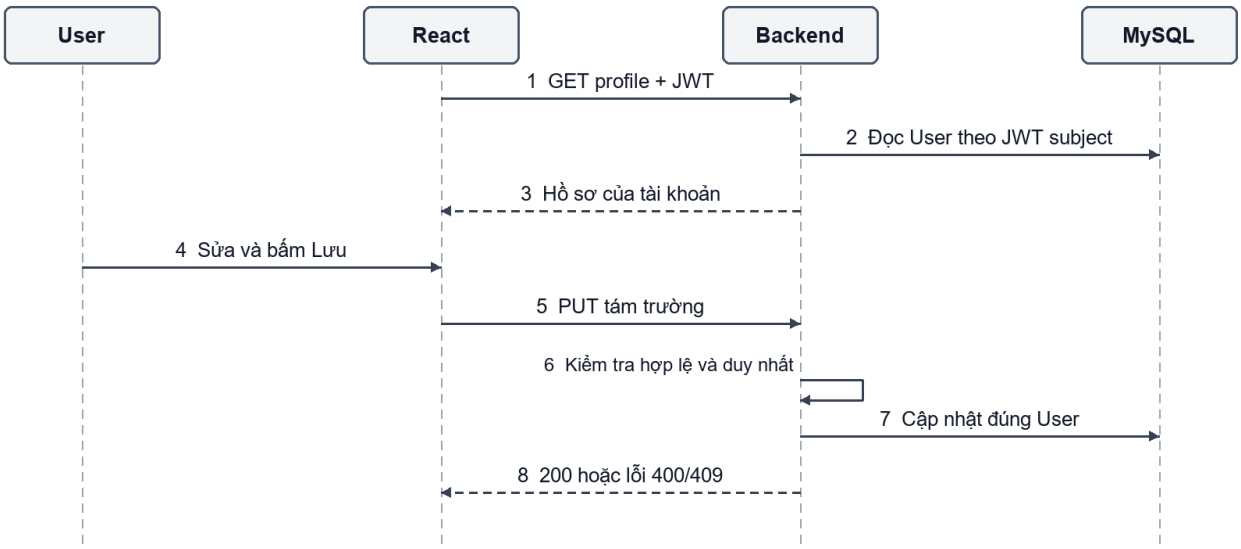
HistoryRow trả id, deviceId, deviceCode, username, action, status, createdAt, confirmedAt, deliveryState. Người thực hiện lấy từ ActionHistory.userId → User.username; người đang xem không thay tác giả. Hai cột Lệnh/Trạng thái không thay cho thông tin deliveryState.

Kiểm tra: ba tài khoản đọc được lịch sử chung, page 5/10/15/20 và các bộ lọc đúng; request 617–622 giữ đúng ba username. Bảng desktop đủ sáu cột, mobile cuộn ngang trong vùng bảng [2].

3.9 Hồ sơ người dùng



Hình 3.18 Giao diện xem hồ sơ và tài nguyên dự án

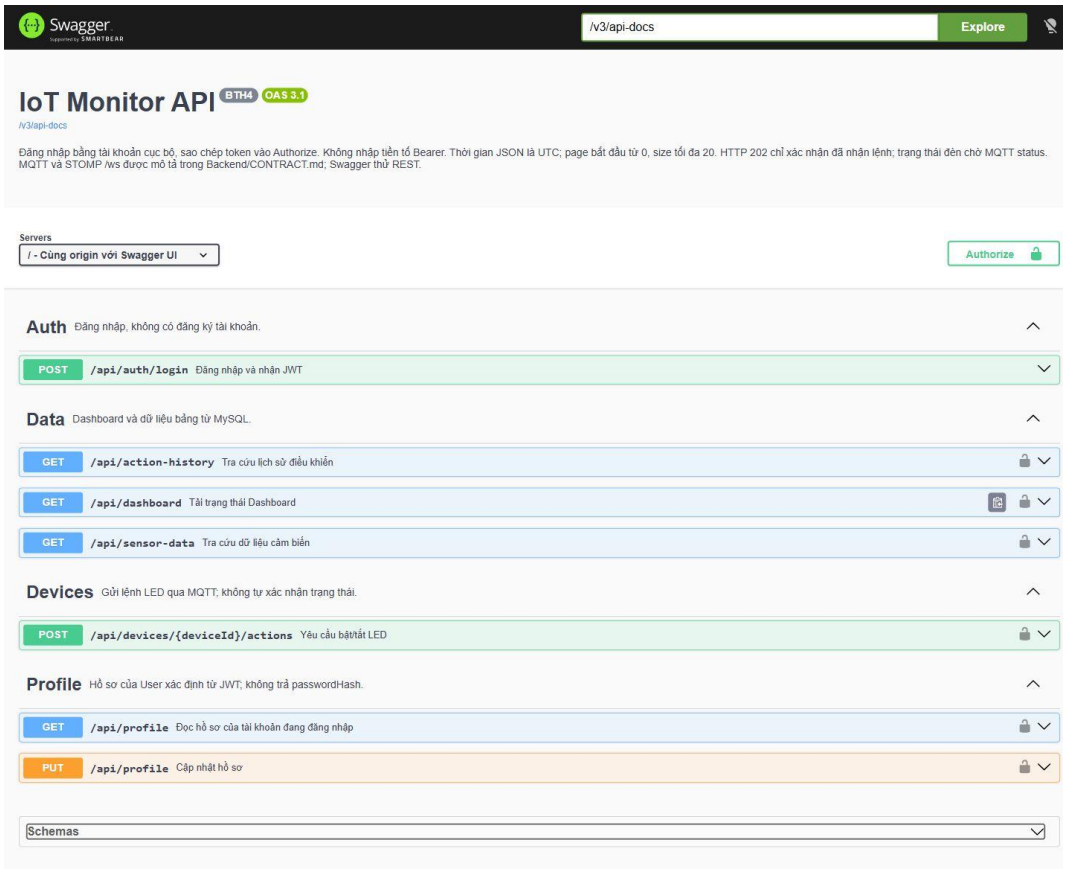


Hình 3.19 Sequence xem và cập nhật hồ sơ

GET /api/profile đọc User theo JWT; PUT /api/profile thay tám trường: fullName, studentCode, email, githubUrl, figmaUrl, apiDocsUrl, reportUrl, avatarUrl. Ba trường đầu bắt buộc; avatar chỉ nhận HTTPS. Không sửa id, username hoặc passwordHash.

Lưu hợp lệ trả 200; dữ liệu sai 400; email/mã sinh viên trùng 409. Hủy bỏ thay đổi chưa lưu. Kiểm tra: PUT tạm user125, GET xác nhận rồi khôi phục tám trường; user123 không đổi. Đã thử email trùng, avatar HTTP và body giả username. updatedAt thay đổi do PUT [2].

3.10 Tài liệu API và vận hành



Hình 3.20 Swagger hiển thị bảy thao tác REST

Swagger UI tại /swagger-ui/index.html, OpenAPI tại /v3/api-docs. Bảy thao tác gồm Login, Dashboard, SensorData, ActionHistory, gửi action và GET/PUT Profile. Schema có HistoryRow.username. API nghiệp vụ dùng bearer JWT; tài liệu công khai không đồng nghĩa API được bỏ xác thực [8].

Khởi động bằng .\start.ps1 ở thư mục dự án; script dùng MySQL container hiện có và Compose app/broker. Kiểm tra docker compose ps, health app và Dashboard. Cấu hình DB/MQTT/JWT nằm trong file môi trường được gitignore; không đưa secrets vào báo cáo hoặc Git.

Dừng app/broker bằng docker compose --env-file Backend/.env stop. Không dùng down -v để tránh mất dữ liệu; không chạy thêm Backend cùng nhận telemetry. Mosquitto Windows không tự tranh cổng 1325. Firmware giữ IP LAN phù hợp; nội bộ Docker dùng tên service [2].

CHƯƠNG 4 KẾT LUẬN

4.1 Kết quả đạt được

Hệ thống đã thực hiện thu thập cảm biến, giám sát realtime, điều khiển LED có xác nhận, tra cứu và quản lý hồ sơ. Ba tài khoản có cùng chức năng, lịch sử lưu đúng người gửi; dữ liệu được quản lý bằng năm bảng và có tài liệu API.

Bảng 4.1 Tổng hợp kết quả

Hạng mục kiểm tra	Kết quả ngày 27/10/2026
Telemetry	522 MQTT reading, 522 STOMP, 174/Sensor; 522 hàng SQL khớp, 0 sai lệch.
LED Online	8 request CONFIRMED: 617–622 và 627–628; đúng thiết bị, action và username.
Offline và timeout	4 request 623–626 không ACK; có timeout, Device giữ OFF. Dashboard tự Online sau cắm lại.
Giao diện	15 điểm mỗi đường thẳng hàng đầu/cuối; tooltip, chú giải, tìm giá trị/thời gian, Processing, màu ON/OFF và reload hoạt động.
API và hồ sơ	142 mục kiểm tra PASS, 0 FAIL, 1 OBSERVED; PUT/khôi phục user125 và các nhánh xác thực đã nêu đạt.
Tự động và build	12 test Frontend, 4 test Backend H2/MQTT mock đạt; Vite và Docker app image build thành công.

Phiên chốt 18:41:54–18:50:36 ngày 27/10/2026 được lưu tại bộ final-review của dự án. Các số lượng trên chỉ thuộc tập đã đối chiếu; 142 mục kiểm tra không phải 142 endpoint độc lập. Test H2/MQTT mock được tách khỏi kết quả ESP32 thật [2].

4.2 Hạn chế

Telemetry chưa có sampleId hoặc timestamp đo tại ESP32; không suy ra ba Sensor cùng chu kỳ từ giờ nhận. Lux chưa hiệu chuẩn; ACK chỉ phản ánh GPIO, chưa có cảm biến xác nhận LED phát sáng độc lập. Chưa kiểm thử broker/Wi-Fi lỗi, ACK muộn/lặp/đảo thứ tự với phần cứng thật, tải lớn hoặc chạy dài hạn; môi trường nội bộ chưa có TLS đầy đủ.

4.3 Hướng phát triển

Bổ sung mã chu kỳ và timestamp đo; hiệu chuẩn ánh sáng; kiểm thử reconnect/ACK bất thường và đo từng chặng truyền. Sau đó hoàn thiện TLS, sao lưu/phục hồi và cơ chế chống ghi trùng nếu mở rộng Backend. Các thay đổi MQTT phải đồng bộ firmware, Backend và Frontend.

TÀI LIỆU THAM KHẢO

[1] Hoàng Quốc Anh. Bài thực hành 1 Hệ thống IoT giám sát môi trường và điều khiển thiết bị. TH1_IOT_V5.docx, 2026.

[2] Mã nguồn dự án và bộ kiểm tra 2026-10-27-final-review: compose.yaml, Backend/CONTRACT.md, schema.sql, firmware IoTMonitor.ino; api-review.json, telemetry-comparison.json, planned-command-results.json và ảnh giao diện. Chốt ngày 27/10/2026.

[3] MQTT.org. MQTT Specifications. https://mqtt.org/mqtt-specification/

[4] Spring. STOMP over WebSocket. https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html

[5] React. useEffect. https://react.dev/reference/react/useEffect

[6] Oracle. MySQL 8.0 Fixed Point Types. https://dev.mysql.com/doc/refman/8.0/en/fixed-point-types.html

[7] Espressif. Arduino ESP32 ADC API. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[8] Swagger. Bearer Authentication. https://swagger.io/docs/specification/v3_0/authentication/bearer-authentication/

Các tài liệu trực tuyến được truy cập ngày 27/10/2026.

PHỤ LỤC TỪ ĐIỂN DỮ LIỆU

Năm bảng gồm 40 cột; kiểu và khả năng NULL được đối chiếu MySQL. PK: khóa chính; FK: khóa ngoại; UQ: duy nhất. DATETIME(6) lưu tới microsecond theo quy ước UTC của ứng dụng.

User

Cột	Kiểu SQL	NULL	Ràng buộc
id	bigint	Không	PK, tự tăng
username	varchar(50)	Không	UQ
passwordHash	varchar(255)	Không	—
fullName	varchar(100)	Không	—
studentCode	varchar(30)	Không	UQ
email	varchar(255)	Không	UQ
githubUrl	varchar(500)	Có	—
figmaUrl	varchar(500)	Có	—
apiDocsUrl	varchar(500)	Có	—
reportUrl	varchar(500)	Có	—
avatarUrl	varchar(500)	Có	—
createdAt	datetime(6)	Không	—
updatedAt	datetime(6)	Không	—

Sensor

Cột	Kiểu SQL	NULL	Ràng buộc
id	bigint	Không	PK, tự tăng
code	varchar(30)	Không	UQ
name	varchar(100)	Không	—
type	varchar(30)	Không	—
unit	varchar(20)	Không	—
active	tinyint(1)	Không	—
createdAt	datetime(6)	Không	—
updatedAt	datetime(6)	Không	—

SensorData

Cột	Kiểu SQL	NULL	Ràng buộc
id	bigint	Không	PK, tự tăng
sensorId	bigint	Không	FK → Sensor.id
value	decimal(12,4)	Không	—
recordedAt	datetime(6)	Không	—

Device

Cột	Kiểu SQL	NULL	Ràng buộc
id	bigint	Không	PK, tự tăng
code	varchar(30)	Không	UQ
name	varchar(100)	Không	—
type	varchar(30)	Không	—
status	varchar(10)	Không	—
active	tinyint(1)	Không	—
createdAt	datetime(6)	Không	—
updatedAt	datetime(6)	Không	—

ActionHistory

Cột	Kiểu SQL	NULL	Ràng buộc
id	bigint	Không	PK, tự tăng
userId	bigint	Không	FK → User.id
deviceId	bigint	Không	FK → Device.id
action	varchar(10)	Không	—
status	varchar(10)	Không	—
createdAt	datetime(6)	Không	—
confirmedAt	datetime(6)	Có	—